

1. RÈGLES DU JEU

Deux joueurs jouent sur une grille carrée, de côté de longueur impaire. Alice part du centre du bord gauche. Bob part du centre du bord du haut. Chaque joueur joue à tour de rôle. Lorsque c'est son tour, un joueur peut se déplacer vers l'une des (au plus) quatre cases adjacentes à sa case actuelle. Le but du jeu est d'être le premier à atteindre son bord opposé, peu importe la case sur ce bord.

Au lieu de se déplacer, à chaque tour un joueur peut décider de poser une barrière sur le plateau. Une barrière est un segment de longueur $2 - \epsilon$ cases. Elle est posée horizontalement ou verticalement entre des cases de la grille. Une fois qu'une barrière est posée, il n'est plus possible de se déplacer en traversant la barrière. Il n'est pas possible de poser une barrière par dessus une autre barrière, même partiellement. Chaque joueur dispose initialement de 10 barrières. Il est interdit de poser une barrière si celle-ci rend impossible la victoire d'un joueur.

Si les deux joueurs sont situés sur deux cases adjacentes, il est possible de se déplacer en sautant au-delà du pion de l'adversaire. Dans ce cas, on peut donc se déplacer sur n'importe quelle case adjacente au pion de l'adversaire.

2. GRAPHE DES COUPS

L'ensemble de toutes les parties possibles dans ce jeu est représenté par un graphe orienté dont les sommets sont les états du plateau et les arêtes vont d'un plateau vers un autre lorsqu'il existe un coup valide d'un joueur entre ces deux états.

On utilise les types OCaml suivants :

OCAML

```
type joueur =  
  | Alice  
  | Bob  
  
(** On numérote les cases à partir du coin en haut à gauche *)  
type coord = int * int  
  
type barriere = {  
  (** Indique si la barrière est horizontale ou verticale *)  
  horizontal : bool;  
  (** Coordonnées de la barrière. On indique en fait la case en dessous à gauche pour  
      une barrière horizontale, ou la case en haut à droite pour une barrière  
      verticale. *)  
  pos : coord;  
}  
  
type plateau = {  
  dim : int;  
  alice : coord;  
  bob   : coord;  
  barrieres : barriere list;  
  stock_alice : int;  
  stock_bob   : int  
}
```

Question 1. Le type `plateau` est incomplet. Quelle information faut-il ajouter pour décrire correctement un sommet du graphe du jeu ? Modifier le type `plateau` pour corriger cela.

Question 2. Écrire une fonction `gagnant` qui indique le gagnant, s'il y en a un, pour un plateau donné.

OCAML

```
val gagnant : plateau -> joueur option
```

Question 3. Écrire une fonction `cases_separees` qui prend en paramètre deux coordonnées de cases, supposées adjacentes dans le plateau, et une barrière et décide si les cases sont séparées par la barrière.

```
val cases_separees : position -> position -> barriere -> bool
```

OCAML

Question 4. Écrire une fonction `affiche_plateau` qui affiche dans le terminal une représentation du plateau de jeu.

Question 5. Il est interdit de poser une barrière si elle chevauche une autre barrière déjà posée. De plus, une barrière doit tenir intégralement dans le plateau. Écrire une fonction `barriere_possible` qui prend en paramètre un emplacement de barrière et un plateau et indique si le placement est possible en respectant cette règle.

```
val barriere_possible : barriere -> plateau -> bool
```

OCAML

Question 6. Écrire une fonction `cases_voisines` qui prend en paramètres un plateau et une position et renvoie la liste des cases voisines accessibles, en tenant compte des barrières et des bords du plateau.

```
val cases_voisines : plateau -> position -> position list
```

OCAML

Question 7. Un joueur peut gagner s'il existe un chemin entre sa position actuelle et l'une des cases de son bord d'arrivée. Écrire une fonction `peut_gagner` qui prend en paramètre un joueur et un plateau et décide si le joueur peut gagner, étant donné le placement des barrières. (Indication : pour cette fonction, il est inutile de prendre en compte le pion adverse, puisqu'il est possible de le sauter en se déplaçant.)

```
val peut_gagner : joueur -> plateau -> bool
```

OCAML

Question 8. Écrire une fonction `coups_possibles` qui prend en paramètre un plateau et renvoie la liste des plateaux suivants possibles en jouant un coup.

```
val coups_possibles : plateau -> plateau list
```

OCAML

3. STRATÉGIE GLOUTONNE

Une stratégie est une fonction qui, à chaque plateau où le joueur doit jouer, associe un coup à jouer pour ce joueur. Dans cette partie, on s'intéresse à un joueur qui ne pose aucune barrière et qui suit simplement le chemin le plus court entre sa position et l'une des cases d'arrivée. S'il existe plusieurs tels chemin, on en choisit un au hasard.

Question 9. Écrire une fonction `plus_court_chemin` qui prend en paramètres la position d'un joueur, une fonction qui décide si une case est une case d'arrivée pour ce joueur, et un plateau et qui renvoie la liste des cases à parcourir sur un plus court chemin du joueur vers l'arrivée.

```
val plus_court_chemin : coord -> (coord -> bool) -> plateau -> coord list
```

OCAML

Question 10. Écrire la fonction `strategie_gloutonne`, uniquement dans le cas où le joueur qui doit jouer est Alice.

```
val strategie_gloutonne : plateau -> plateau
```

OCAML

Question 11. La stratégie gloutonne ne pose pour l'instant aucune barrière. À chaque coup, on pourrait plutôt, si le joueur possède encore une barrière à poser :

- calculer la longueur d'un plus court chemin gagnant pour le joueur,
- calculer la longueur d'un plus court chemin gagnant pour l'adversaire,

- si elle est strictement plus petite pour l'adversaire, essayer de poser une barrière sur un plus court chemin de l'adversaire,
- sinon, avancer le pion du joueur.

Modifier la fonction `strategie_gloutonne` pour obtenir une fonction `strategie_gloutonne_barrieres` qui implémente ce comportement.

4. ÉLAGAGE $\alpha\beta$

Question 12. Commencer par programmer min-max et vérifier que cela marche sur des très petits plateaux.

Question 13. Quels paramètres faut-il ajouter pour modifier ce min-max en élagage $\alpha\beta$?

Question 14. On suppose que l'on dispose déjà d'une heuristique donnée, qui évalue le score attribué à un plateau (sur un ensemble de valeurs de votre choix). Programmer l'algorithme d'élagage $\alpha\beta$ vu en cours, en limitant la recherche avec une profondeur donnée en paramètre.

Question 15. En déduire un programme intéressant qui vous permet de gagner la compétition de Quouloir. Le serveur impose une limite de temps, vous ne pouvez pas prendre plus de quelques secondes pour trouver un coup à chaque fois.

5. JOUER EN RÉSEAU

On souhaite faire une compétition de jeux de Quouloir où vos programmes jouent les uns contre les autres. Pour cela, on dispose d'un serveur arbitre dont le rôle est d'organiser des parties entre vos programmes. La communication avec le serveur s'effectue via le réseau, au travers d'une *socket*, qui se manipule quasiment comme un fichier.

Le protocole entre un client et le serveur est le suivant. Dans les exemples, on préfixe visuellement par $\leftarrow$ les messages envoyés par le client et par $\rightarrow$ ceux envoyés en réponse par le serveur, mais ces flèches ne sont pas partie des données échangées sur la socket. Chaque message est écrit sur une ligne qui se termine par le caractère `\n`. Chaque ligne contient des données au format JSON, sans aucun retour à la ligne.

J'ai commencé à rédiger le fichier `joueur.ml` pour vous aider à discuter avec le serveur.

Question 16. Terminer l'implémentation pour faire un programme qui se connecte au serveur et joue à Quouloir en respectant le protocole de communication. L'adresse du serveur pouvant changer, votre programme prendra cette adresse en paramètre sur la ligne de commande. Ainsi, pour lancer votre programme nommé `joueur`, on s'attend à exécuter dans le shell l'appel :

```
./joueur adresse_serveur
```

CONSOLE